

LABORATORIO 2

Análisis

Requisitos

funcionales

1. Al iniciar el programa se deben solicitar los datos del festival como nombre, código y nombre del coordinador.
2. El festival puede contener un máximo de 5 escenarios almacenados en un arreglo básico.
3. Cada escenario debe contener información de mínimo el nombre, código, ubicación, capacidad máxima de asistentes y estado.
4. Se debe evaluar que la capacidad máxima de asistentes sea mayor que 0, de lo contrario, lanzar una `IllegalArgumentException`.
5. Al registrar un nuevo escenario se debe indicar en que posición del arreglo se quiere agregar. Esto conlleva las validaciones de que esté dentro de los límites del arreglo, que esté disponible y que la información sea válida.
6. Cuando no se haya configurado un arreglo se debe tener el valor `null`.
7. El programa debe permitir consultar los escenarios que ya se hayan configurado anteriormente y manejar la situación en caso no haya escenarios.
8. Por medio de la posición se deberá poder consultar un escenario específico y se debe validar que la posición no esté vacía o sea inválida.
9. Se debe permitir modificar la capacidad máxima de asistentes y el estado de un escenario ya configurado/registrado. Debe también validar que los valores sean válidos y que no se intente modificar una posición vacía.
10. El programa debe permitir retirar/eliminar un escenario, colocando la posición en `null`.
11. Los artistas deben ir almacenados en un `ArrayList` al ser posible una cantidad desconocida de artistas.

12. Cada artista debe contener la información de código, nombre artístico, genero musical, duración de la presentación en minutos y la cantidad estimada de asistentes.
13. La duración de la presentación debe ser mayor que 0 y la cantidad estimada de asistentes no puede llevar valores negativos. De no cumplirse, lanzará una `IllegalArgumentException`.
14. Se debe poder registrar a nuevos artistas en el `ArrayList` media vez no se repita el código del artista.
15. El programa debe permitir consultar los artistas que ya se hayan registrado anteriormente y manejar la situación cuando no existan artistas.
16. Se debe poder encontrar un artista por medio de su código al recorrer el `ArrayList`.
17. Se debe permitir modificar la información de un artista cuando este ya este registrado, validando la nueva información.
18. El programa debe permitir eliminar un artista del `ArrayList`, cancelando su participación.
19. El programa debe poder calcular y mostrar los siguiente: cantidad de escenarios configurados, cantidad de espacios disponibles, escenario con mayor capacidad máxima, cantidad de artistas registrados, artista con la presentación de mayor duración, artista con mayor cantidad estimada de asistentes, y el promedio de duración de las presentaciones.
20. Los cálculos de artista deben considerar los datos que ya estén registrados en el `ArrayList` para evitar operar en una lista vacía.
21. Cuando se ingrese un valor distinto al solicitado en Scanner se debe manejar una `InputMismatchException` de manera que el sistema continúe.
22. Se deben utilizar try-catch para el manejo de excepciones en el programa.
23. Se debe utilizar al menos un finally en una operación con sentido.
24. El programa no debe terminar de manera inesperada, debe seguir funcionando a pesar de entradas incorrectas o excepciones manejadas.

Clases

Main: Será el punto de inicio al instanciar el controlador, iniciando su ejecución.

Controlador: Será el cerebro del programa, conectando a Festival con Vista, interpretando la opción que se elija del menú, solicitando y entregando datos. También maneja las excepciones.

Vista : Presentará las opciones al usuario, pedirá datos y mostrará mensajes y errores. Se encarga de la interacción con el usuario.

Festival: Contendrá toda la información relacionada con el festival como el nombre, código y nombre del coordinador de información propia y se encargará de gestionar el CRUD de artistas y escenarios.

Escenario: Contendrá la información del escenario.

Artista : Contendrá la información del artista.

Tabla de propiedades

#	Tipo de datos	Nombre	Visibilidad	Motivo	Clase
1	Scanner	sc	Private	Permitir que el usuario pueda ingresar datos	Vista
2	String	nombreFest	Private	Nombre del festival	Festival
3	int	codigoFest	Private	Código del festival	Festival
4	String	nombreCoor	Private	Nombre del coordinador del festival	Festival
5	Escenario[]	escenarios	Private	Almacenar a los escenarios (Max 5)	Festival
6	ArrayList<Artista>	artistas	Private	Almacenar a los artistas del festival sin un numero definido	Festival
7	int	codigoEsc	Private	Código del escenario	Escenario
8	String	nombreEsc	Private	Nombre del escenario	Escenario
9	String	ubicación	Private	ubicación del escenario	Escenario

10	int	capMaxAsist	Private	Capacidad máxima de asistentes	Escenario
11	bool	estado	Private	Estado del escenario(disponible/ no disponible)	Escenario
12	int	codigoArt	Private	Código del artista	Artista
13	String	nombreArt	Private	nombre artístico	Artista
14	String	generoMus	Private	genero musical del artista	Artista
15	int	duracionMin	Private	duración de la presentación del artista en minutos	Artista
16	int	cantidadEst	Private	Cantidad estimada de asistentes	Artista
17	Vista	vista	Private	instanciar vista	Controlador
18	Festival	festival	Private	instanciar el festival	Controlador

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetros	Motivo	Clase
1	Void	main()	Public	String [] args	Punto de inicio main	Main
2		Festival	Public	String nombreFest, int codigoFest,	Constructor del festival, también inicializa	Festival

				String nombreCoor	Escenario[] y ArrayList<Artista>	
3	String	getNombreFest	Public		obtener el nombre del festival	Festival
4	int	getCodigoFest	Public		Obtener el código del festival	Festival
5	String	getNombreCoor	Public		Obtener el nombre del coordinador del festival	Festival
6	void	nuevoEscenario	Public	int posicion, int codigoEsc, String nombreEsc, String ubicacionEsc, int capMaxAsist, bool estado	Guardar un nuevo escenario dentro del arreglo fijo cuando hay espacio y su capacidad es mayor que 0	Festival
7	Escenario[]	consultaEscenari os	Public		Arreglo de los escenarios registrados	Festival
8	Escenario	consultaEscenari oEsp	Public	int posicion	Información de un escenario especifico	Festival

9	void	modificarEscenario	Public	int posición, int capMaxAsist, bool estado	Modificar la cantidad máxima de asistentes y el estado del escenario	Festival
10	void	eliminarEscenario	Public	int posicion	Eliminar un escenario de acuerdo a su posición en el arreglo	Festival
11	int	numEscenarios	Public		Cuantificar los escenarios registrados	Festival
12	int	espaciosDisponibles	Public		Cuantificar los espacios libres (null) dentro del arreglo	Festival
13	Escenario	escMayorCapacidad	Public		Encontrar el escenario que tiene mayor capacidad máxima	Festival
14	void	nuevoArtista	Public	int codigoArt, String nombreArt, String generoMusc, int duracionMin, int cantidadEst	registrar un nuevo artista cuando el código es distinto, la duración es mayor que 0 y la cantidad estimada es mayor o igual que 0	Festival
15	ArrayList<Artista>	consultarArtistas	Public		Consultar el ArrayList de artistas registrados	Festival

16	Artista	consultarArtista	Public	int códigoArt	Busca el artista según su código y manda null si no existe	Festival
17	void	modificarArtista	Public	int codigoArt, String nombreArt, String generoMusc, int duracionMin, int cantidadEst	Modifica los datos del artista, menos su código, ya que es unico	Festival
18	void	eliminarArtista	Public	int codigoArt	Elimina un artista al buscarlo por su código	Festival
19	int	numArtistas	Public		Cuantificar los artistas registrados	Festival
20	Artista	mayorDuracionArt	Public		Encontrar el artista con la presentación de mayor duración	Festival
21	Artista	mayorCantidadEst	Public		Encontrar el artista que registro mayor cantidad estimada de asistentes	Festival
22	double	promedioDuracion	Public		Calcular el promedio de la duración de las presentaciones.	Festival

23		Escenario	Public	int codigoEsc, String nombreEsc, String ubicación, int capMaxAsist, bool estado	Constructor del escenario	Escenari o
24	int	getCodigoEsc	Public		Obtener el código del escenario	Escenari o
25	String	getNombreEsc	Public		Obtener el nombre del escenario	Escenari o
26	String	getUbicacionEsc	Public		Obtener la ubicación del escenario	Escenari o
27	Int	getCapMaxAsist	Public		Obtener la capacidad máxima del escenario	Escenari o
28	bool	getEstado	Public		Obtener el estado del escenario	Escenari o
29	void	setCapMaxAsist	Public	int capMaxAsist	Modificar la capacidad máxima de asistentes	Escenari o

30	void	setEstado	Public	bool estado	Modificar el estado del escenario	Escenario
31		Artista	Public	int códigoArt, String nombreArt, String generoMusc, int duracionMin, int cantidadEst	Constructor del artista	Artista
32	int	getCodigoArt	Public		Obtener el código del artista	Artista
33	String	getNombreArt	Public		Obtener el nombre artístico del artista	Artista
34	String	getGeneroMusc	Public		Obtener el genero musical del artista	Artista
35	int	getDuracionMin	Public		Obtener la duración en minutos de la presentación	Artista
36	int	getCantidadEst	Public		Obtener la cantidad estimada de asistentes	Artista

37	void	setNombreArt	Public	String nombreArt	Establecer un nuevo nombre artístico	Artista
38	void	setGeneroMusc	Public	String generoMusc	Establecer un nuevo genero musical	Artista
39	void	setDuracionMin	Public	int duracionMin	establecer nueva duración de la presentacion	Artista
40	void	setCantidadEst	Public	int cantidadEst	establecer nueva cantidad estimada de asistentes	Artista
41	void	iniciar()	Public		iniciar el controlador	Controlador
42	void	nuevoFestival	Public		Solicitar la información del festival	Controlador
43	void	configurarEscenario	Public		Pedir la configuración de un escenario nuevo. Busca excepciones	Controlador
44	void	consultarEscenarios	Public		consultar los escenarios configurados	Controlador

45	void	consultarEscenario	Public		consultar la información de un escenario específico. Valida si existe	Controlador
46	void	modificarEscenario	Public		modificar un escenario	Controlador
47	void	retirarEscenario	Public		eliminar un escenario del arreglo	Controlador
48	void	registrarArtista	Public		Registrar un nuevo artista en el ArrayList. Busca excepciones	Controlador
49	void	consultarArtistas	Public		consulta los artistas registrados	Controlador
50	void	buscarArtista	Public		Buscar un artista por su código	Controlador
51	void	modificarArtista	Public		modificar la información de un artista al solicitar el código	Controlador
52	void	cancelarParticipacion	Public		Eliminar la participación de un artista	Controlador

53	void	mostrarReporteFest	Public		Mostrar el reporte del festival: escenarios configurados y espacio disponibles, el escenario con mayor capacidad, la cantidad de artistas registrados, el artista con la presentación de mayor duración, el artista con mayor cantidad estimada de asistentes y el promedio de duración de las presentaciones.	Controlador
54	void	salir	Public		Terminar la ejecución del programa	Controlador
55	int	mostrarMenu()	Public		Mostrar el menú	Vista
56	int	pedirDatosNumericos	Public	String texto	Pedir un dato numérico al usuario. Maneja el InputMismatchException	Vista
57	String	pedirTexto	Public	String texto	Pedir un dato en texto al usuario	Vista

58	void	mensaje()	Public	String texto	mostrar un mensaje determinando el resultado de la operación realizada	Vista
59	void	error()	Public	String texto	Mostrar un mensaje cuando hay un error	Vista
60	void	mostrarEscenarios	Public	Escenario [] escenarios	Mostrar los escenarios registrados	Vista
61	void	mostrarEscenarioEsp	Public	Escenario e, int posicion	Mostrar un escenario específico	Vista
62	void	mostrarArtistas	Public	ArrayList<Artista> artistas	Mostrar los artistas registrados	Vista
63	void	mostrarArtista	Public	Artista artista	Mostrar un artista buscado por su código	Vista
64	void	mostrarReporte	Public	int numEscenarios, int espaciosDisponibles, Escenario escMayorCapacidad, int numArtistas, Artista artista mayorDuracionA	Muestra el reporte del festival	Vista

				rt, Artista mayorCantidad Est, double promedioDuraci on		
--	--	--	--	---	--	--

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad identificada es escenarios del tipo Escenarios[], almacenará objetos de la clase Escenario y tendrá un tamaño de 5 objetos fijos.

4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad identificada es Artistas, ya que nuevos artistas pueden incorporarse, modificar los datos de su presentación o cancelar su participación. Almacenará objetos de la clase Artista.

7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales serán proveídos a través de los constructores que reciben los datos desde la vista por medio de Scanner. Se validarán los valores numéricos tal como la capacidad máxima, duración de la presentación que sean mayores que 0 y cantidad estimada de asistentes que sea mayor o igual que 0. De no cumplirse se generará un IllegalArgumentException.

8. ¿Cómo determinará si una posición del arreglo contiene un escenario o contiene null?

Por medio de comprobaciones dentro de ifs que evalúe si la posición seleccionada es igual a null o retorna algún valor.

9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Para realizar estas operaciones se realizará un ciclo for para comparar el código del artista por el que se ha buscado. En caso de querer modificarlo, se busca por código y se mandan a llamar los setters destinados a las propiedades que pueden ser modificadas. Para eliminarlo se puede utilizar un método de ArrayList .remove().

10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Las situaciones que pueden producir excepciones son:

IllegalArgumentException cuando se ingresen valores negativos, menores o iguales a 0, a excepción de cantidad de asistentes estimados del artista que puede ser mayores o iguales que 0, pueden suceder en nuevoEscenario, modificarEscenario, nuevoArtista y modificarArtista. IndexOutOfBoundsException cuando se consulta un índice fuera del rango del arreglo, puede producirse en nuevoEscenario, consultaEscenarioEsp, modificarEscenario y eliminarEscenario. NullPointerException cuando se consulta una posición del arreglo vacía, debe evaluarse que la posición no sea null antes. Y también InputMismatchException cuando se ingresan valores que no corresponden a los solicitados en Scanner, como pedirDatosNumericos.

Se utilizarán los try-catch en el programa por medio de controlador en la mayoría de las excepciones y en vista en el caso de InputMismatchException. En el caso de NullPointerException se validará primero si el escenario es igual a null. Finally será utilizado aun así se ejecute una excepción o no, tal como al finalizar el programa.